BioSync: Make Steps Match Google Fit Without Opening It

Oct 31, 2026 · @R

BioSync already reads Health Connect and runs a step-sensor bridge, so the design is right. The number still lags because the bridge cannot recover steps taken before its first sensor reading of the day, and a 500 ms polling loop probably starves Health Connect. Below: six causes with file and line, then the fix. I read the code at commit 6570f0f but could not build or run it here, so test every change on a phone.

What the code does today

`BioSyncHealthStepManager.kt` shows Fit's flushed steps from Health Connect plus new sensor steps since an anchor: `steps = hcFitSteps + (lastRawCounter - hcAnchorRawCounter)`. The anchor is set the first time BioSync sees the sensor that day, not at midnight. The file's own comment (L148–151) assumes Fit only flushes to Health Connect when opened, so BioSync fills the gap from the phone's step sensor. That idea is right. It fails because the anchor starts too late.

The app uses Health Connect only, so the end-of-2026 shutdown of the Google Fit APIs does not affect it. My first draft of this doc assumed the Fit REST API; this version replaces it.

Six causes

#	What goes wrong	Where	Effect on your number
1	The bridge cannot count steps from before its first reading of the day. A fresh process sets the anchor to the current counter. If the service was killed overnight (common on Xiaomi, Oppo, Vivo, Realme) or the phone rebooted, steps since Fit's last flush are never counted.	`BioSyncHealthStepManager.kt` L550–561, L411–417	BioSync stays behind Fit until Fit is opened and flushes. This is your symptom.
2	After a Fit flush, if you are not walking, the anchor snaps to the latest counter and the total may drop to Fit's flushed value.	L951–962	BioSync can sit below Fit's live number after every flush.
3	Polling storm: a 500 ms loop runs in `rememberBioSyncStepSyncController`, which three screens create, plus a 1–2 s loop in the service. Each tick does 2 `readRecords` (pages of 5000 and 3000) and 2 aggregates. Failures are swallowed by `runCatching` and `getOrDefault(0)`.	L1556–1560; `MainActivity.kt` L378; `DashboardScreen.kt` L406; `ProtocolSummaryScreen.kt` L357; service L68–75; swallowed at L1107, L1178, L1251	Health Connect throws `RateLimitExceededException` when an app calls too often. I cannot see your Logcat, so confirm this one, but it would give stale data and no visible error.

#	What goes wrong	Where	Effect on your number
4	Two threads write the same state: sensor callbacks run on the main thread, Health Connect reconcile runs on IO, both touching `hcAnchorRawCounter`, `lastRawCounter` and `pendingDetectorSteps`.	L539–599 vs L935–984	Occasional jumps up or down.
5	`startForeground` is wrapped in `runCatching`, so a failure is silent and the service runs as a plain background service that Android kills. The "all permissions" gate also ignores background-read and notification permission.	`BioSyncStepBackgroundService.kt` L101–115; manager L125–128	The service dies quietly and live updates stop.
6	BioSync requests `WRITE_STEPS` and `WRITE_DISTANCE` but never writes steps, and runs a `deleteRecords` purge on first sync.	manifest L10, L12; manager L240–243, L1058–1072	A heavier permission screen and a harder Google Play health review, for no benefit.

The fix: count from a saved day baseline

The phone's step counter (`TYPE_STEP_COUNTER`) keeps counting while BioSync is dead, and Google Fit reads the same sensor. So today's steps can be computed from the sensor alone, as long as the app remembers the counter value at the start of the day. Health Connect is then only a correction, not the main source.

1. **Baseline:** the counter value at 00:00, saved to disk. The day total is `lastRaw - baselineRaw`.
2. **Carry:** if the phone reboots, the counter restarts near 0. Steps counted before the reboot go into `carry`.
3. **Offset:** if Fit reports more steps than the sensor total (a watch, or a gap BioSync missed), the difference is saved once and added on top. It only grows when Fit is ahead, so BioSync never drops below Fit's flushed value.
4. **Undercount beats overcount:** when the midnight value is unknown, the baseline is set to the current counter. The total is then too low, and Fit's next flush repairs it through the offset. An overcount could not be repaired.

Replace `hcFitSteps`, `hcAnchorRawCounter`, `pendingDetectorSteps` and `reconcileHealthConnectWithHardwareBridge` with this ledger. It is plain Kotlin with no Android dependencies, so you can unit-test it.

```
data class StepLedger(
  val dayKey: String = "",
  val bootCount: Int = -1,
  val baselineRaw: Float = -1f,   // sensor counter at the start of today,
this boot
  val carry: Int = 0,             // steps counted today before the last
```

```
reboot
  val lastRaw: Float = -1f,
  val lastRawAtMs: Long = 0L,
  val offset: Int = 0,            // steps Fit has that the sensor did not
see
)

object StepLedgerMath {
  fun sensorPart(l: StepLedger): Int =
    max(0f, l.lastRaw - l.baselineRaw).roundToInt() + l.carry

  fun total(l: StepLedger): Int = sensorPart(l) + l.offset

  fun onRaw(
    l: StepLedger, raw: Float, nowMs: Long,
    today: String, startOfTodayMs: Long, boot: Int, bootTimeMs: Long,
  ): StepLedger {
    // If the phone booted before today began, steps since boot include
yesterday's: undercount instead.
    val freshBaseline = if (bootTimeMs >= startOfTodayMs) 0f else raw
    return when {
      l.dayKey.isEmpty() ->            // first run ever
        StepLedger(today, boot, raw, 0, raw, nowMs)
      l.dayKey != today -> {           // new day
        val sameBoot = l.bootCount == boot
        val nearMidnight = startOfTodayMs - l.lastRawAtMs <= 3_600_000L
        val base = if (sameBoot && nearMidnight) l.lastRaw else if (sameBoot)
raw else freshBaseline
        StepLedger(today, boot, base, 0, raw, nowMs)
      }
      l.bootCount != boot ->           // reboot today: counter restarted
near 0
        l.copy(carry = sensorPart(l), bootCount = boot, baselineRaw =
freshBaseline,
               lastRaw = raw, lastRawAtMs = nowMs)
      raw < l.lastRaw -> l             // ignore a glitchy reading
      else -> l.copy(lastRaw = raw, lastRawAtMs = nowMs)
    }
  }

  fun onFitTotal(l: StepLedger, fitSteps: Int): StepLedger {
    val s = sensorPart(l)
    return if (fitSteps > s + l.offset) l.copy(offset = fitSteps - s) else l
  }
}
```

Get the boot values from the system: `Settings.Global.getInt(context.contentResolver, Settings.Global.BOOT_COUNT, -1)` for `boot`, and `System.currentTimeMillis() - SystemClock.elapsedRealtime()` for `bootTimeMs`. Save the whole ledger in `BioSyncLocalStore` in place of the `stepSensor*` and `lastHealthConnectWrittenSteps` fields.

The shown number is `StepLedgerMath.total(ledger)`. Distance can stay as `steps × metersPerStep`, as it is now.

Wire it in, in this order

1. **Stop the polling (cause 3).** In `rememberBioSyncStepSyncController`, delete the `while (isActive) { delay(500L) ... }` loop at L1556–1560. Keep the single `syncWithHealthConnect` call on entry. Delete the 1–2 s `syncWithHealthConnect` loop in the service (L68–75) and use the loop in step 3.
2. **One cheap Health Connect read, with errors logged.** Replace both `readRecords` calls and the fallback origins with one aggregate:

```
suspend fun fitStepsToday(client: HealthConnectClient): Int? = runCatching {
  val zone = ZoneId.systemDefault()
  val start = LocalDate.now(zone).atStartOfDay(zone).toInstant()
  val r = client.aggregate(AggregateRequest(
    metrics = setOf(StepsRecord.COUNT_TOTAL),
    timeRangeFilter = TimeRangeFilter.between(start, Instant.now()),
    dataOriginFilter = setOf(DataOrigin(GOOGLE_FIT_PACKAGE)),
  ))
  r[StepsRecord.COUNT_TOTAL]?.toInt()
}.onFailure { Log.w("BioSync", "Health Connect read failed", it)
}.getOrNull()
```

3. **Service loop that matches the new design.** The sensor callback updates the ledger instantly; Health Connect is read at most once a minute.

```
syncLoopJob = serviceScope.launch {
  while (isActive) {
    BioSyncHealthStepManager.probeHardwareStepCounterNow(applicationContext)
    BioSyncHealthStepManager.refreshFitTotalIfDue(applicationContext,
minIntervalMs = 60_000L)
    delay(15_000L)
  }
}
```

4. **One writer for the ledger (cause 4).** Every update goes through one lock, so the main-thread sensor callback and the IO-thread Health Connect read cannot interleave:

```
private val ledgerLock = Any()
@Volatile private var ledger = StepLedger()

private fun updateLedger(ctx: Context, f: (StepLedger) -> StepLedger) =
synchronized(ledgerLock) {
  ledger = f(ledger)
  BioSyncLocalStore.saveLedger(ctx, ledger)       // write this: stores all
StepLedger fields
  _syncState.value = _syncState.value.copy(
    liveTodaySteps = StepLedgerMath.total(ledger), lastSyncedEpochMillis =
System.currentTimeMillis())
}
```

5. **A catch-up worker, so the number is right even if the service was killed (cause 1).** WorkManager cannot start a foreground service from the background, but it does not need to: it reads the sensor once, reads Health Connect once, saves, and exits.

```
// build.gradle.kts: implementation("androidx.work:work-runtime-ktx:<latest
stable>")
class StepCatchUpWorker(ctx: Context, p: WorkerParameters) :
CoroutineWorker(ctx, p) {
  override suspend fun doWork(): Result {
    BioSyncHealthStepManager.catchUpOnce(applicationContext) // one-shot
sensor read + one aggregate + save
    return Result.success()
  }
}

WorkManager.getInstance(ctx).enqueueUniquePeriodicWork(
  "biosync-step-catchup", ExistingPeriodicWorkPolicy.KEEP,
  PeriodicWorkRequestBuilder<StepCatchUpWorker>(15,
TimeUnit.MINUTES).build())
```

Also have the worker schedule a one-time job for the next 00:00, so the midnight baseline is saved close to midnight. Android may delay both jobs under battery saver; opening BioSync still probes the sensor immediately, so the number is correct on screen.

6. **Make failures visible (cause 5).** Replace the `runCatching` around `startForeground` with `try/catch` that logs and calls `stopSelf()`. Add notification permission (Android 13+) and, when `FEATURE_READ_HEALTH_DATA_IN_BACKGROUND` is available, background-read permission to `isAllPermissionsAllowed`, and show a "Reconnect" banner when either is missing. Show "Last synced HH:MM" from `lastSyncedEpochMillis`.

7. **Remove the write permissions (cause 6).** Delete `WRITE_STEPS` and `WRITE_DISTANCE` from the manifest and from `buildRequestedHealthPermissions` and `baseHealthPermissions`, change `isHealthConnectAllowAllGranted` to require read only, and delete the `deleteRecords` block at L1058–1072.
8. **Battery.** Register `TYPE_STEP_COUNTER` with `SENSOR_DELAY_NORMAL` instead of `SENSOR_DELAY_FASTEST`; the ledger only needs the latest value, and the one-shot probe already fetches it on demand. Register `TYPE_STEP_DETECTOR` only while a BioSync screen is visible, and drop `pendingDetectorSteps`.

Test plan

Before changing code, add a debug line to Logcat on every ledger update: time, raw counter, baseline, carry, offset, Fit total, shown total. It shows which cause is hitting you, and `RateLimitExceededException` in Logcat would confirm cause 3.

Test	What to do	Pass if
Closed-app catch-up	Force-stop BioSync and Google Fit, walk 100 steps, open only BioSync	BioSync shows about +100 within 2 seconds
Match with Fit open	Open Google Fit and BioSync side by side, three times in a day	Difference is 5 steps or fewer
Killed service	Swipe BioSync from recents, wait 30 minutes, walk 50 steps, open BioSync	Total includes the 50 steps
Reboot	Note the total, reboot, walk 50 steps, open BioSync	Previous total plus about 50 (steps during shutdown are lost)
Midnight	Leave the phone idle across 00:00, check before walking	Starts at 0 to 5, never carries yesterday's steps
Watch or second source	Walk with a Wear OS watch paired to Fit, then open Fit once	BioSync catches up to Fit after the flush and keeps counting from there

Limits you should know

- **"Exact" means within a few steps.** Fit shows its own live read of the sensor. BioSync reads the same sensor, but the two reads are seconds apart, and Fit can add steps from a watch that the phone never saw. The offset repairs that only after Fit flushes to

Health Connect.

- **Android 13 and below** needs the Health Connect app installed; Android 14 and above has it built in.
- **Aggressive phone makers** can still stop the service. The worker and the resume probe keep the on-screen number right, but a closed app cannot update a notification. Ask users to allow auto-start and unrestricted battery for BioSync.
- **I have not compiled this.** The ledger is plain Kotlin and easy to unit-test first: feed it a sequence of raw readings across a reboot and a midnight and check the totals.

Next step, if you want it: I can write the exact `saveLedger` and migration code against `BioSyncLocalStore.kt`, so the new fields drop in without breaking saved data.